

Test Strategy

What the Workbench is, how it is built, and how QA proves it behaves, one architectural layer at a time. For each layer: what can go wrong there, which mechanism catches it, where that check runs, and what is deliberately left out.

OWNER	VERSION	BASELINE	BOARD
Shanthi Subramanian · BA / QA	2.0 · 24 September 2026	release/1.00 @ 9f6a3be	Azure DevOps · Epics 1– 19

01 The project

What is under test, who uses it, and the rules it must never break.

The client is a large national bank. Before any business unit launches a product, adds a feature, changes a process, onboards a vendor, enters a geography or opens to a new customer segment, its Financial Crimes Risk Management (FCRM) function must assess the financial crime risk. Today that runs on email, Word, Excel and SharePoint, takes **15–20 business days**, produces inconsistent conclusions between analysts, and leaves examiners reconstructing reasoning by hand.

The **Risk Assessment Workbench** carries a change request from intake to a committee decision on one governed platform. AI prepares: it maps the request to FFIEC risk categories, extracts structure from submitted documents and drafts the assessment narrative. Humans decide at every step.

STAGE	ACTOR	WHAT HAPPENS	STATUS AFTER
Intake	Product Owner	Raises one of six change types, attaches documents, links a mock CRM / core-banking / vendor record	Submitted
Assessment	FCRM Analyst	Reviews AI category mapping, extracted fields, policy reliance and drafted narrative; scores; overrides with a reason; finalizes	InAssessment
Committee	Committee Member	Votes on the routed, read-only assessment until quorum resolves it	PendingCommittee
Decision	System records	Approved · Rejected · Deferred · Approved with conditions, pushed back to the source system	Decisioned
Configuration	Admin	Tunes scoring, workflow rules and (Epic 19) SLA targets and breach distribution lists, always with a reason	–

The rules the product is judged on

These are the reason the test strategy is shaped the way it is. Each is a *must never*, and each needs proof at a layer that cannot be bypassed.

- 1. **The system prepares, humans decide.** Nothing is auto-approved, auto-rejected, or allowed past a review gate unreviewed.
- 2. **Residual risk never reaches zero**, at calculation time and at configuration time.
- 3. **Any AI output can be overridden, with a stated reason**, and both the original and the override are kept.
- 4. **The audit trail is append-only at the data layer**, not merely hidden from the UI.
- 5. **Risk categories come from a real, citable framework** (FFIEC BSA/AML), never invented by the model.
- 6. **Synthetic data only.** No connection to a real bank system, and no real mailbox.

46 API ENDPOINTS · 13 CONTROLLERS	23 DB TABLES · 46 STORED FUNCTIONS	3 AI TOUCHPOINTS	4 ROLES	19 EPICS ON THE BOARD
---	--	--	-------------------------------	---

02 In scope & out of scope

Stated explicitly, so an untested area is a decision on record rather than a surprise.

IN SCOPE – QA OWNS

- Functional behaviour of Epics 1–10 and 14 against their acceptance criteria
- API contract, validation, error mapping and workflow-order rules on all 46 endpoints
- Database-enforced governance: audit trigger, CHECK constraints, reason guards
- The three role journeys end to end in the browser
- AI output quality for category mapping, extraction and narrative (evaluation harness)
- Human-in-the-loop gates: every AI output reaches the committee only through a review
- Role-based access (Epic 11) at the API, not just the UI
- Examiner-ready audit export (US-9.3), content and completeness
- Mock-system integration and behaviour when a mock system is down
- SLA tracking and breach email (Epic 19), as it is built
- Deployment smoke checks on the dev Container Apps environment
- Security gates in the pipeline: dependency, static and image scans (reviewing results)

OUT OF SCOPE – AND WHY

- **Writing unit tests** — Dev owned; QA consumes the result (39 test methods today)
- **Load and performance at bank scale** — a hackathon workload; only response-time sanity is observed
- **Penetration testing** — automated scanners only; no adversarial engagement
- **Real integrations** — CRM, core banking and vendor systems are mocks by rule
- **Real email delivery** — breach mail is captured in a local inbox, never sent
- **Auth0, Azure AI Foundry and Azure platform behaviour** — trusted as managed services; only our integration with them is tested
- **Model internals** — we evaluate outputs, not weights, training or provider latency
- **Cross-browser and mobile** — Chromium desktop only
- **Disaster recovery, backup restore, localisation**
- **Infrastructure-as-code correctness** — owned by DevOps through the Terraform PR pipeline

THE RULE THAT PREVENTS DUPLICATED EFFORT

Every check happens at the cheapest layer that can actually see it. API tests never re-assert scoring arithmetic — unit tests do that in milliseconds. UI tests never check a status code. Each layer only covers what the layers beneath it structurally cannot reach.

03 Architecture through a tester's eyes

The same diagram the developers know, annotated with where each kind of test enters and what each layer can get wrong.

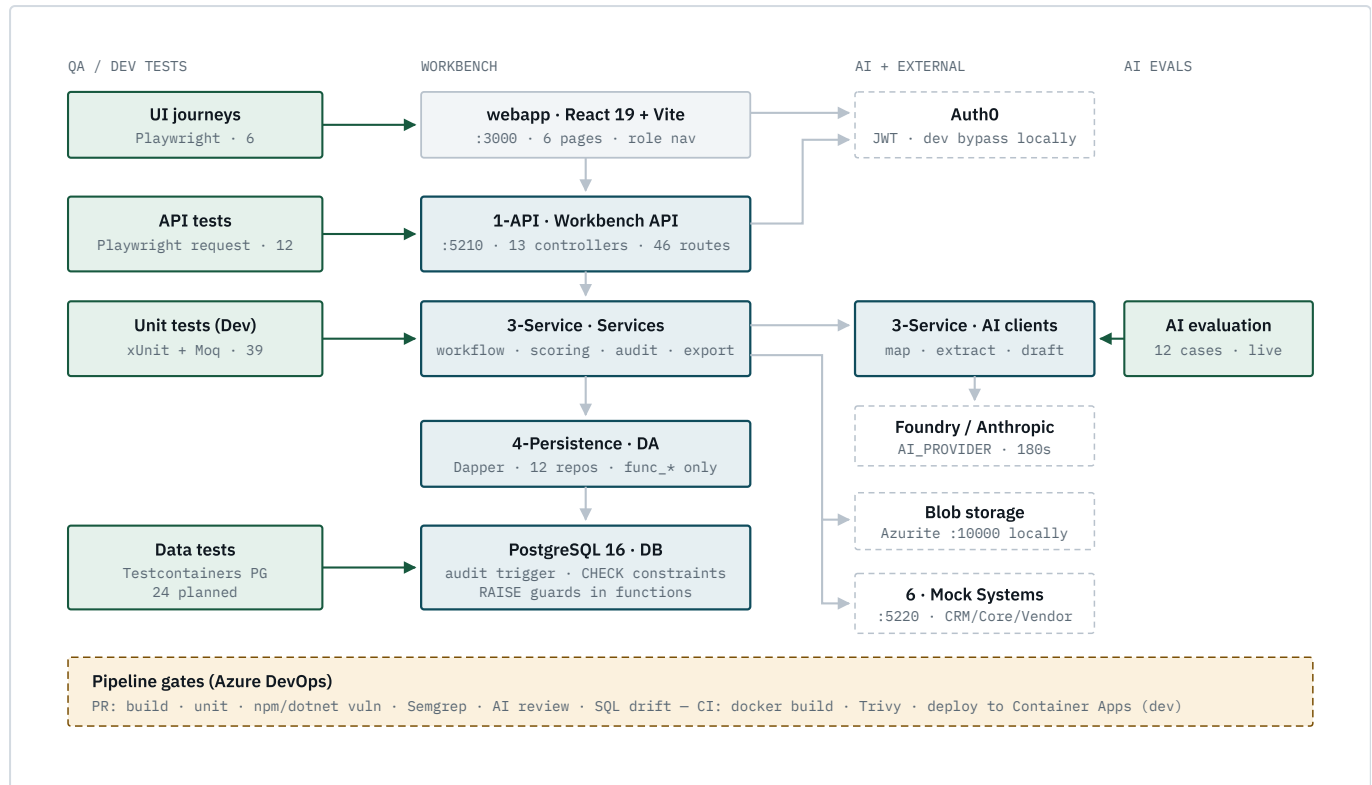


Figure 1. Five test entry points plus the pipeline. The lowest entry point matters most for governance: the append-only audit trigger and the risk-score constraints live in PostgreSQL, and no HTTP endpoint ever attempts to alter an audit row — so only a direct database connection can prove the database refuses one.

What each layer can get wrong

Read from the code on `release/1.00`, not from documentation. The right-hand column is the finding that shapes how the layer is tested.

LAYER	RESPONSIBILITY	CHARACTERISTIC FAILURE	OBSERVED IN CODE
webapp 5-Presentation	Screens per role, state-switched (no router), Auth0 login, dev user switcher	A screen hidden by role is mistaken for access control; a gate shown as ready when it is not	Role only hides navigation; DEF-004 shows "Ready to finalize" on an empty assessment
1-API Web	Controllers, auth scheme, DI, <code>OperationResult</code> envelope	Wrong status codes; missing authorization; caller able to name the actor	Bare <code>[Authorize]</code> , no role policy; <code>actorUserId</code> taken from the body; any exception becomes 500

LAYER	RESPONSIBILITY	CHARACTERISTIC FAILURE	OBSERVED IN CODE
3-Service Services	Workflow order, scoring, committee resolution, ingestion, audit export	Transitions in the wrong order; a downstream outage breaks intake	Mock-system calls catch <code>HttpRequestException</code> only — a timeout escapes
3-Service AI	Prompting, JSON parsing, grounding against allowed categories	Fabricated category or citation; malformed JSON; provider down	Unknown category ids dropped; parse failure falls back to empty; provider outage returns 500, no retry
4-Persistence DA	Dapper repos calling stored functions only	Unmapped database errors surface as 500	FK violation and unique-key race (DEF-006) both reach the caller as 500
DB PostgreSQL	Schema, 46 functions, triggers, constraints, seeds	A governance rule enforced only above the database	Audit trigger, <code>residual_rating > 0</code> , <code>mitigation < 1.0</code> , override-reason CHECK all present; <code>deploy_all.sql</code> not idempotent
6 • Mock Systems	Synthetic CRM, core banking and vendor registry	Contract drift between mock and Workbench; outage handling	No auth, no custom timeout (100 s default), separate schema in the same database
Pipelines & ops	PR gates, image build and scan, deploy to dev	A green pipeline that ran no behavioural test	CI runs no tests; only the PR pipeline runs unit tests; no health endpoint beyond Ping

04 How each layer is validated

For every layer: what is checked, by which mechanism, where it runs, and who owns it.

Presentation — webapp

WHAT IS VALIDATED	MECHANISM	RUNS	OWNER
The three role journeys complete end to end on one request: raise → assess and finalize → vote and decide	Playwright UI, serial, one browser context per role	Local now; merge pipeline (target)	QA
Gates refuse visibly: Finalize refuses an unreviewed assessment; an unreviewed assessment cannot be routed	Playwright UI, expected-to-fail until DEF-004 closes	Same	QA
Error messages are plain language and never show stack traces	Assertions inside the journeys; exploratory	Same	QA
Accessibility of the Assessment Workspace	Manual spot-check (keyboard, contrast, screen reader)	Before demo	QA
Build and dependencies	<code>npm ci + build + npm audit --audit-level=high</code>	Every PR	Pipeline

API — 1-API

The workhorse layer. Four questions for every write endpoint:

1. **Happy path** — valid input gives the correct result inside the `OperationResult` envelope.
2. **Bad input** — a blank reason, an out-of-range value, an unsupported file or a malformed GUID is refused with a 4xx, never a 500.
3. **Wrong person** — a Product Owner cannot vote, finalize or change configuration; the actor comes from the token, never the body.
4. **Wrong time** — a vote before routing or a route before finalize is refused.

WHAT IS VALIDATED	MECHANISM	RUNS	OWNER
Envelope, seeded roles, FFIEC framework present	Playwright <code>request</code> — smoke (3)	Local; PR (target)	QA
Authentication and role enforcement (Epic 11)	Playwright <code>request</code> — 7 tests, 4 expected-to-fail recording DEF-001/002	Same	QA

WHAT IS VALIDATED	MECHANISM	RUNS	OWNER
Error mapping: FK violation and unknown ids return 4xx	Playwright <code>request</code> — 2 tests, both expected-to-fail (currently 500)	Same	QA
Controller-to-service wiring	Covered implicitly by the above; no separate test	—	—

Services — 3-Service

WHAT IS VALIDATED	MECHANISM	RUNS	OWNER
Scoring arithmetic and bounds; residual never zero	xUnit + Moq, <code>[Theory]</code> tables (ScoringServiceTests)	Every PR	Dev
Committee quorum and resolution precedence: Reject > Defer > Conditions > Approve	xUnit (CommitteeServiceTests, 9)	Every PR	Dev
Intake validation; ingestion snapshot; mock outage does not fail intake	xUnit with mocked <code>IMockSystemsClient</code>	Every PR	Dev
Mock-system <i>timeout</i> (not just refusal) is survived	New QA case: API test with the mock stopped / slowed	Planned	QA
Finalize refuses while any section is still <code>AiDrafted</code> or lacks policy reliance	Unit (Dev) + UI tripwire (QA)	PR / local	Dev + QA

AI — 3-Service AI

Two different questions, answered by two different mechanisms. **Does our code handle whatever the model returns?** is deterministic, so it is a unit test with a fake model. **Is what the model returns good enough?** is probabilistic, so it is an evaluation against fixed datasets.

WHAT IS VALIDATED	MECHANISM	RUNS	OWNER
Parsing, fallbacks, dropping categories not in the allowed list, empty input	xUnit with <code>FakeChatCompletionClient</code> (11 tests across the three clients)	Every PR	Dev
Category mapping: correct FFIEC set, zero fabricated codes	Eval — exact set-overlap, 5 cases	On demand; nightly (target)	QA
Extraction: expected values found, low-confidence fields flagged <code>needsReview</code>	Eval — substring match + flag check, 4 cases	Same	QA

WHAT IS VALIDATED	MECHANISM	RUNS	OWNER
Narrative: cites something, well-formed, honours regeneration feedback	Eval — labelled heuristic, 3 cases	Same	QA
Whether a citation is apposite and the reasoning sound	Manual review by a domain reader	Before demo	QA
Every AI write is persisted as <code>AiProposed</code> / <code>AiExtracted</code> / <code>AiDrafted</code> with a <code>system/AI</code> audit entry	Data test on the audit table	Planned	QA

Data access and database — 4-Persistence

The only layer where the governance rules are *guaranteed* rather than coded. Data tests connect straight to a throwaway PostgreSQL 16 container loaded with the real `deploy_all.sql` and try to break each rule directly.

WHAT IS VALIDATED	MECHANISM	RUNS	OWNER
<code>UPDATE</code> and <code>DELETE</code> on <code>audit_event</code> are rejected by <code>trg_block_audit_event_mutation</code>	Testcontainers data test	Planned · PR	QA
<code>residual_rating = 0</code> , <code>mitigation_factor = 1.0</code> and an override without a reason are refused by CHECK constraints	Same	Same	QA
The five reason guards in stored functions raise on blank reasons	Same	Same	QA
Each state change writes its audit row in the same transaction (rollback leaves neither)	Same	Same	QA
Config versioning: a change deactivates the old row and inserts a new one; assessments keep their pinned version	Same	Same	QA
Generated <code>deploy_all.sql</code> matches the schema sources	SQL drift check (<code>git diff --exit-code</code>)	Mock-API PR	Pipeline

Mock Systems — 6

WHAT IS VALIDATED	MECHANISM	RUNS	OWNER
Lookups return the seeded synthetic records with the fields the framework scores	API test against :5220	Planned	QA
The Workbench snapshot at intake matches what the mock returned at that moment	API test, then compare snapshot	Planned	QA

WHAT IS VALIDATED	MECHANISM	RUNS	OWNER
The committee decision is pushed back as a risk flag	API test reading the mock after a decision	Planned	QA
Data is reproducible from the seed	Re-seed and compare	Planned	QA

Deployment and operations

WHAT IS VALIDATED	MECHANISM	RUNS	OWNER
Image has no HIGH / CRITICAL vulnerabilities	Trivy 0.50.2, fails the job	Every CI run	DevOps
Both services answer after a deploy	Smoke: <code>/api/Ping</code> and <code>/api/ping</code> on the dev Container Apps	Post-deploy (target)	QA + DevOps
The deployed configuration is the intended one	Manual first-deployment walkthrough	Per new environment	QA
Telemetry reaches Application Insights when the connection string is set	Manual query in the workspace after a smoke run	Per environment	DevOps

05 Test mechanisms

The tools, and the four practices that make their results trustworthy.

MECHANISM	TOOL	LOCATION	COUNT TODAY	NEEDS RUNNING
Unit	xUnit 2.9 · Moq 4.20	tests/... UnitTests	39	Nothing
API	Playwright <code>request</code>	tests/e2e/api	12	API + DB
UI journeys	Playwright Chromium	tests/e2e/ui	6	webapp + API + mock + DB
Data	Testcontainers PostgreSQL 16	tests/... DataTests	0 of 24	Docker
AI evaluation	.NET console harness	evals/	12	Live model endpoint
Static & dependency security	Semgrep · dotnet vuln check · npm audit	.azure- pipelines	—	Pipeline
AI code review	Claude review of the PR diff, blocking issues fail	.azure- pipelines	—	Pipeline
Manual	Written charter + recorded outcome	docs/qa	6	A person

1 · We break the code on purpose to check the tests notice

A passing test proves nothing until it has been seen to fail for the right reason. Before any governance test is trusted, the rule it guards is inverted locally and exactly that test must go red. Done for the finalization gate: disabling it failed **exactly five tests** and left the rest green. That selectivity is the signal.

2 · Known gaps are recorded as tests, never deleted

Where the product does not yet do the right thing, the test asserts the **correct** behaviour and is marked `test.fail()`. The board stays readable, the requirement is never softened to match the bug, and the day a fix lands the runner reports *"expected to fail, but passed"* — the fix announces itself. Ten of the eighteen QA tests are such tripwires today.

3 · A dead environment can never look like a pass

An expected-to-fail test also "passes" when the server is down. So `global-setup.js` confirms the API answers before any test runs and aborts loudly if it does not. A suite run against nothing must never produce a green board.

4 · Every test names the story it proves

```
test('US-8.2 AC5 – a vote is recorded against the member by name', ...)
```

Both runners write these names into their results, so traceability back to the Azure DevOps board comes out of a build, not a spreadsheet. A story with no matching test shows up as an empty row on its own.

DETERMINISTIC VERSUS PROBABILISTIC

Anything with one right answer is tested with an exact assertion: scoring, quorum, SLA due dates, constraints. Only model output is evaluated statistically, and the narrative eval is labelled a *heuristic* because it checks form rather than judgement. No LLM is used to grade anything a plain assertion can decide.

06 Governance rules & their proof

Each non-negotiable, where it is enforced, and which test would catch it being broken.

RULE	ENFORCED IN	PROVED BY	CONFIDENCE
Humans decide — nothing reaches committee unreviewed	Service finalize gate; <code>func_routeToCommittee</code> requires Finalized	Unit + UI tripwires (DEF-004)	 MEDIUM
Residual never zero	Formula bound, C# guard, CHECK <code>> 0</code> , config CHECK <code>< 1.0</code>	Unit + API + Data (planned)	 HIGH
Override needs a reason; both values kept	Five function guards + CHECK on risk score	Unit for one of five; Data for all five (planned)	 MEDIUM
Audit append-only	<code>trg_block_audit_event_mutation</code>	Data test (planned) — the only layer that can	 LOW
Citable framework only	AI client drops unknown ids; FFIEC seed	Unit + eval (zero fabricated codes)	 HIGH
Role owns the action	Not yet enforced server-side	API tripwires (DEF-001/002)	 LOW
SLA breach only informs (Epic 19)	To be built	API: breach leaves status, assessment and decision unchanged	NOT BUILT

 **High** — proved at more than one layer, including one that cannot be bypassed. ·  **Medium** — proved at one layer, or with a known gap. ·  **Low** — little or no automated proof today.

Epic 19 — how SLA tracking will be tested

Designed now so the stories are built testable. The confirmed rules — business days, an Admin-maintained holiday calendar, defaults of 2 / 8 / 5 / 15, and the clock pausing during a Product Owner clarification — all have exact answers.

- **Due-date arithmetic (unit):** a table of submit times across weekends, holidays and clarification pauses, each with its expected due date. Time is injected through `TimeProvider`, so no test waits on a real clock.
- **Configuration (API + data):** zero, negative and blank targets refused; a reason required; a new version written; an in-flight request keeps its pinned version.
- **Breach email (API):** mail is captured by a local Mailpit inbox and read back through its API. Exactly one email per request per stage; the content carries no scoring rationale.
- **Humans decide (API):** after a breach, the request's status, assessment and committee decision are byte-for-byte unchanged.

- **Audit (data):** SlaBreached and NotificationSent rows are appended, never updated.

07 Environments & test data

Where tests run, and the data they run against.

ENVIRONMENT	COMPOSITION	USED FOR
Local	docker-compose: PostgreSQL 16 on :5433, Azurite on :10000; API :5210, Mock :5220, webapp :3000; dev auth bypass	All QA layers during development
PR pipeline	Hosted agent; .NET 10, Node 22; Postgres service container (target)	Build, unit, security; API and data tests (target)
Dev (Azure)	Container Apps <code>ca-gh-hrg-workbench-dev</code> and <code>ca-gh-hrg-mockapi-dev</code> , PostgreSQL Flexible Server, Auth0	Post-deploy smoke, manual walkthrough, demo

- **All data is synthetic.** Seeds provide six users covering four roles, the FFIEC framework, a policy corpus, workflow rules, and mock records (six customers, five products, four vendors).
- **Each test creates what it needs** and never depends on another test's leftovers, except the three role journeys, which deliberately share one request because that is the story being proven.
- **Resets are explicit:** `ops/setup-local-db.sh --reset` rebuilds the local database; data tests get a fresh container per run.
- **AI evals use fixed datasets** in `evals/datasets/`, versioned with the code, so a score change is attributable to a prompt or model change rather than to data drift.

08 Pipeline integration

What runs automatically today, and what should.

CHECK	TODAY	TARGET	BLOCKS?	TIME
WORKBENCH PR (INTO RELEASE/1.00)				
Build API + webapp	RUNNING	Every PR	Yes	~2 min
Unit tests (trx published)	RUNNING	Every PR	Yes	~1 min
dotnet vuln check · npm audit	RUNNING	Every PR	Yes	<1 min
Semgrep	RUNNING	Every PR	Informational	~1 min
AI code review	RUNNING	Every PR	On blocking issues	≤15 min
API tests	NOT WIRED	Every PR	Yes	~3 min
Data tests	NOT BUILT	Every PR	Yes	~3 min
MOCK API PR				
SQL drift check · build · vuln · Semgrep · AI review	RUNNING	Every PR	Yes	~5 min
CI ON RELEASE/* AND LATER				
Docker build · push · Trivy · deploy to dev	RUNNING	On merge	Yes	~8 min
Post-deploy smoke	NOT WIRED	After deploy	Yes	<1 min
UI journeys	NOT WIRED	On merge	Soft, then hard	~5 min
AI evaluation	NOT WIRED	Nightly	Report only	~3 min

PR feedback stays under fifteen minutes, dominated by the AI review. A gate slower than that gets bypassed, and a bypassed gate is worse than none because it creates false confidence.

STATED PLAINLY

The CI pipelines that deploy to dev run **no behavioural tests**, and no pipeline collects coverage. A green deploy today means "it built, it scanned clean and it started", not "it works". Wiring the API suite and a post-deploy smoke closes most of that gap for little effort.

09 Entry, exit & defects

When a story is ready to test, when it is done, and what happens when it is not.

GATE	CRITERIA
Entry — a story is testable	Acceptance criteria in Given/When/Then on the board · merged to <code>release/1.00</code> · PR pipeline green · seed data covers the scenario
Exit — a story is done	Every criterion has a named test or a recorded manual outcome · no open Critical or High defect against it · governance rules it touches proved at the data layer where one exists
Exit — the release	Three role journeys pass on dev · all expected-to-fail tripwires either still failing for the recorded reason or promoted to passing · eval run recorded in <code>evals/results/</code> · six manual scenarios signed off

Severity

LEVEL	MEANING	EXAMPLE
CRITICAL	A governance rule can be broken	An unreviewed assessment reaches the committee
HIGH	A role cannot complete its job, or data is exposed	User directory readable without a credential
MEDIUM	Wrong behaviour with a workaround	Unknown category returns 500 instead of 400
LOW	Cosmetic or wording	Inconsistent label

Defects are raised on the Azure DevOps board with a reproduction, the story they break and the test that covers them. A criterion is only added to a story when it states behaviour the system should have and does not — never to describe what the code happens to do.

Manual scenarios

SCENARIO	WHY A PERSON
AI narrative quality	The eval checks form; only a domain reader judges whether a citation is apposite
Examiner-readiness of the audit export	"Suitable for an examiner" is judged by reading it
Exploratory testing of overrides	Free-text reasons; scripts find only what they were told to

SCENARIO	WHY A PERSON
First deployment walkthrough	Only a person spots a present-but-wrong setting
Accessibility spot-check	Scanners catch roughly a third of real problems
Committee decision semantics	Whether the flow makes sense to people deliberating

Each gets a written outcome: date, who ran it, what they found. A manual pass with no note is indistinguishable from one that never ran.

10

Risks & current position

What could undermine this strategy, and the next five things, in order.

18	10	39	12	24
QA TESTS AUTOMATED	OF WHICH TRIPWIRES	DEV UNIT TESTS	AI EVAL CASES	DATA TESTS TO BUILD

RISK	EFFECT ON TESTING	MITIGATION
Auth is bypassed locally and roles are not enforced	Role tests can only record the gap	Tripwires stay until Epic 11 lands; rerun against Auth0 on dev
AI evals need a live, paid endpoint	Evals skip silently when not configured	Treat "Skip" as not run, never as pass; nightly job with a configured key
<code>deploy_all.sql</code> is not idempotent	Re-running setup on a used database fails	Fresh container per data-test run; <code>--reset</code> locally
Six days to submission	Not everything planned will be built	Order below is by governance value, not effort

#	NEXT	GIVES US	OWNER
1	Build the audit-immutability and constraint data tests	The one rule only the database can prove moves from Low to High	QA
2	Wire the API suite into the Workbench PR pipeline	12 tests start protecting every change	DevOps + QA
3	Add a post-deploy smoke to CI	A green deploy means the services answer	DevOps
4	Run the evals against the live model and commit the result	AI quality evidence for the 10% evaluation criterion	QA + Dev
5	Execute and record the six manual scenarios	Judgement checks done before the demo	QA

THE STRATEGY IN ONE SENTENCE

We test each thing once, at the cheapest layer that can see it; we prove governance rules at the layer that cannot be bypassed; we prove our tests work by breaking the code on purpose; and where we are not yet confident, this document says so rather than rounding up.

Component names, ports, counts, constraints and pipeline steps were read from `release/1.00` at commit 9f6a3be and from the QA suite on `qa/ba-qa-workstream`, not from documentation. Counts are as of 24 September 2026. Supersedes version 1.0 (21 September 2026). Companion documents: Test Starter Guide, QA Test Case Register, Business Requirement Coverage.